МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО”

Факультет прикладної математики
Кафедра програмного забезпечення комп’ютерних систем

Лабораторна робота № 4

з дисципліни “ Прикладне програмне забезпечення мовою Python”
тема “Розробка програм з використанням ООП”

Виконав
Студент III курсу
групи КП-31
Власенко Артем

Київ 2026

Варіант 2

1. Написати клас, який реалізує патерн *Singleton.*

2. Написати клас *ORMMapper*, який у конструкторі приймає ім'я іншого існуючого класу, який має лише поля і за допомогою методу *convert_to_db()* створює на основі структури отриманого класу таблицю в базі даних.

3. Написати клас, який містить два методу "*get_string*" і "*print_upper_string*", де перший метод приймає рядок, а другий виводить даний рядок в верхньому регістрі.

4. Написати клас Counter, що реалізує десятковий лічильник, який може збільшувати або зменшувати значення на одиницю в заданому діапазоні, включаючи межі діапазону. У класі мають бути передбачені такі можливості: конструктор для ініціалізації лічильника значеннями за умовчанням (стартове значення, нижня та верхня межі діапазону), метод для його ініціалізації з довільними значеннями, а також методи для збільшення та зменшення поточного значення лічильника. Усі методи класу повинні приймати лише іменовані параметри та перевіряти вихід поточного значення лічильника за допустимий діапазон. Створіть екземпляр лічильника зі значеннями за замовчанням та виведіть на екран його початкові параметри. Далі перевірте його роботу циклом у межах діапазону, збільшуючи та виводячи на екран його поточне значення від мінімально можливого до максимального. Потім перевизначте лічильник, задавши негативну нижню та позитивну верхню межі, а також встановивши позитивне стартове значення для відліку. Знову ж таки, перевірте його роботу циклом, зменшуючи та виводячи на екран його поточне значення від стартового до мінімально можливого. Вкажіть свідомо більшу кількість ітерацій циклів в обох випадках, забезпечивши переривання їх роботи при спробі виходу лічильника за межі діапазону.

```python
import sqlite3
from typing import get_type_hints


class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance


class ORMMapper:
    _type_map = {str: "TEXT", int: "INTEGER", float: "REAL"}

    def __init__(self, cls):
        self._cls = cls

    def convert_to_db(self):
        hints = get_type_hints(self._cls)
        if not hints:
            hints = {
                k: type(v)
                for k, v in vars(self._cls).items()
                if not k.startswith("_") and not callable(v)
            }
        columns = ", ".join(
            f"{name} {self._type_map.get(typ, 'TEXT')}"
            for name, typ in hints.items()
        )
        sql = f"CREATE TABLE {self._cls.__name__} ({columns});"
        print(sql)
        conn = sqlite3.connect(":memory:")
        conn.execute(sql)
        conn.commit()
        conn.close()
```

```python
class StringHolder:
    def __init__(self):
        self._string = ""

    def get_string(self, s):
        self._string = s

    def print_upper_string(self):
        result = ""
        for x in self._string:
            if 'a' <= x <= 'z':
                x = chr(ord(x) - 32)
            result += x
        print(result)
```

```python
class Counter:
    def __init__(self, *, start=0, lower=0, upper=9):
        self._apply(start=start, lower=lower, upper=upper)

    def reinitialize(self, *, start, lower, upper):
        self._apply(start=start, lower=lower, upper=upper)

    def increment(self):
        if self.value >= self.upper:
            raise ValueError(f"Upper bound {self.upper} reached")
        self.value += 1

    def decrement(self):
        if self.value <= self.lower:
            raise ValueError(f"Lower bound {self.lower} reached")
        self.value -= 1

    def _apply(self, *, start, lower, upper):
        if lower > upper:
            raise ValueError("lower must be <= upper")
        if not (lower <= start <= upper):
            raise ValueError("start must be within [lower, upper]")
        self.value = start
        self.lower = lower
        self.upper = upper
```

```python
if __name__ == "__main__":
    # Task 1
    s1 = Singleton()
    s2 = Singleton()

    # "is" checks if both variables point to the same object in memory
    print(s1 is s2)    # True


    # Task 2
    class Person:
        name: str
        age: int
        height: float

    mapper = ORMMapper(Person)
    mapper.convert_to_db()
    # will print: CREATE TABLE Person (name TEXT, age INTEGER, height REAL);


    # Task 3
    holder = StringHolder()
    holder.get_string("hello world")
    holder.print_upper_string()    # HELLO WORLD
```

```python
# Task 4
counter = Counter()
print(f"value={counter.value}, lower={counter.lower}, upper={counter.upper}")

steps = []
for _ in range(15):
    try:
        counter.increment()
        steps.append(counter.value)
    except ValueError:
        break
print(steps)    # [1, 2, 3, 4, 5, 6, 7, 8, 9]

counter.reinitialize(start=5, lower=-3, upper=10)
print(f"value={counter.value}, lower={counter.lower}, upper={counter.upper}")

steps = []
for _ in range(15):
    try:
        counter.decrement()
        steps.append(counter.value)
    except ValueError:
        break
print(steps)    # [4, 3, 2, 1, 0, -1, -2, -3]
```

Висновки

Виконуючи дану лабораторну роботу я ознайомився з основними принципами об'єктно-орієнтованого програмування та їх реалізацією в Python. Я навчився створювати класи з конструкторами, методами та полями різних рівнів доступу, а також застосовувати інкапсуляцію, спадкування і поліморфізм на практиці. Окрім цього, я реалізував патерн Singleton, розробив клас Counter з перевіркою меж діапазону та клас ORMMapper, що динамічно генерує SQL-таблиці на основі структури переданого класу. Таким чином, використання ООП дозволяє створювати більш структурований, гнучкий та повторно використовуваний код, що є важливою навичкою при розробці сучасного програмного забезпечення.

Питання для самоперевірки

1. ООП — це парадигма програмування, де програма будується з об'єктів, які поєднують дані та методи роботи з ними.
2. ООП у Python дозволяє організовувати код у вигляді класів та об'єктів, що спрощує його повторне використання, розширення та підтримку.
3. Основні компоненти ООП у Python — це класи, об'єкти, інкапсуляція, спадкування та поліморфізм.
4. Клас — це шаблон для створення об'єктів. Створюється за допомогою ключового слова class, після якого вказується ім'я класу та двокрапка.
5. Екземпляр класу створюється викликом імені класу з круглими дужками: obj = MyClass().
6. Метод **init** є конструктором класу і викликається автоматично при створенні екземпляра для ініціалізації його атрибутів.
7. Перший аргумент методів класу — це self, який містить посилання на поточний екземпляр об'єкта.
8. Статичний метод створюється декоратором @staticmethod і не приймає self чи cls. Метод класу створюється декоратором @classmethod і приймає cls як перший аргумент.
9. Для цього перед ім'ям атрибута ставляться два підкреслення: self.__attr. Python застосує механізм name mangling, що унеможливить прямий доступ ззовні класу.
10. Функція property() призначена для створення керованих атрибутів через геттер, сеттер і делітер. Приймає аргументи fget, fset, fdel та doc. Може використовуватись як декоратор @property.
11. Python шукає успадковані атрибути відповідно до порядку MRO (Method Resolution Order), послідовно перевіряючи поточний клас і всі батьківські класи.
12. З підкласу до методу суперкласу можна звернутися через вбудовану функцію super(): super().method().

13. Композиція — це принцип побудови класу шляхом включення об'єктів інших класів як атрибутів, замість використання спадкування.
14. Магічні методи визначають поведінку об'єктів при стандартних операціях (порівняння, арифметика, виведення тощо). Від звичайних методів вони відрізняються подвійним підкресленням на початку та в кінці імені: **str**, **len** тощо.
15. Деструктор **del** викликається інтерпретатором автоматично коли на об'єкт не залишається жодних посилань і він підлягає збірці сміття, або при завершенні програми.